

Generative AI

Assignment 2 Dates Generator

1. Problem Description

The task is to build a conditional date generator that takes four constraints as input and produces a valid calendar date that satisfies them. These constraints are the weekday, the month, whether the year is a leap year, and the required decade. For example, the decade value **181** means that the generated year should be between 1810 and 1819. The output date is written in the format **d-m-yyyy**. This problem is treated as a generation task rather than a classification task because the same input conditions can have many correct dates. For that reason, exact-match accuracy is not a suitable metric. Instead, I use Constraint Satisfaction Rate (CSR), which measures whether the generated date satisfies all four conditions at the same time. The dataset covers all dates from 1 January 1800 to 31 December 2200, giving about 146,000 date-condition pairs.

2. Tokenization Design

Designing the tokenizer was the most critical architectural decision. The wrong tokenization can make the problem unsolvable regardless of how powerful the model is.

Condition encoding

Each of the four input conditions is mapped to an integer index from a fixed vocabulary. The day vocabulary has 7 tokens (MON through SUN), the month vocabulary has 12 tokens, the leap vocabulary has 2 tokens (True, False), and the decade vocabulary has 41 tokens covering the integers 180 through 220. The full condition vocabulary is therefore $7 + 12 + 2 + 41 = 62$ tokens. Each input sample is represented as a vector of 4 integer indices.

Date encoding : digit by digit

The most important design choice was to encode dates digit by digit rather than as whole integers or as string tokens. A date like 10-1-1810 is first zero-padded to the fixed format DDMMYYYY, giving 01011810, and then split into 8 individual digit characters, each encoded as an index from 0 to 9. Two special tokens are prepended and appended: a start token **<S>** and an end token **<E>**. The full digit vocabulary is therefore 13 tokens (0–9, **<S>**, **<E>**, **<P>**).

The reason for digit-by-digit encoding is that the model must learn positional structure. The first two positions are always the day (01–31), the next two are the month (01–12), and the last four are the year (1800–2200). If we treated the year as a single token, the model would need to predict from a vocabulary of over 400 values, which is both harder and less interpretable. Breaking it into digits allows the model to learn that position 5 (year digit 1) is almost always 1 or 2, that position 6 depends on the decade condition, and so on

3. Models

Four models were implemented. Two are from the course and two are from outside it.

3.1 Conditional Autoencoder : in-course

Why we chose it: The AE was covered in the course and gives a clean deterministic baseline. It is the simplest model in the set, which makes it useful for understanding what the problem looks like before adding complexity.

The encoder takes the four condition tokens, embeds each one, and flattens them into a single vector. This vector passes through a three-layer MLP and is compressed into a bottleneck latent vector z . The decoder then takes z concatenated with the original condition embeddings and predicts logits for all 8 digit positions at once. The skip connection from the condition embeddings to the decoder is important — the bottleneck alone can lose fine-grained condition detail, so giving the decoder direct access to the original embeddings keeps the full condition signal available.

Loss function:

$$L = (1/8) \times \text{sum over } t \text{ of CE(logits_t , target_t)}$$

Plain cross-entropy averaged over the 8 digit positions. No regularisation term.

The core limitation is that the AE is fully deterministic. For the same four conditions it always outputs the same date. This means it cannot capture the fact that many valid dates exist for the same input, and that is exactly what motivated the other three models.

3.2 Conditional GAN with WGAN-GP : in-course (required)

Why we chose it: GANs were covered in the course and are required by the assignment. A GAN is a natural fit here because the noise vector z directly models the one-to-many nature of the problem : different z samples for the same condition produce different valid dates.

The generator takes the condition embeddings concatenated with a random noise vector sampled from $N(0, 1)$ and passes them through a three-layer MLP with batch normalisation to produce logits for the 8 digit positions. The discriminator takes the condition embeddings concatenated with a flattened one-hot representation of the date and outputs a scalar score.

Loss function (WGAN-GP)

Discriminator loss:

$$L_D = E[D(\text{fake})] - E[D(\text{real})] + \text{lambda} \times \text{GP}$$

Generator loss:

$$L_G = -E[D(G(z, cond))]$$

Gradient penalty:

$$GP = E[(\| \text{grad } D(\hat{x}) \|_2 - 1)^2] \text{ where } \hat{x} = \alpha \times \text{real} + (1 - \alpha) \times \text{fake}, \alpha \sim \text{Uniform}(0, 1)$$

WGAN-GP was chosen over the standard GAN for three concrete reasons. First, it removes mode collapse, the Wasserstein distance is a more meaningful measure of how far two distributions are apart. Second, the discriminator loss actually correlates with generation quality, so it can be used to monitor training instead of just watching for collapse. Third, there are no vanishing gradients from sigmoid saturation because the discriminator outputs a raw score, not a probability.

The discriminator uses spectral normalisation and no batch normalisation. Batch normalisation breaks the gradient penalty computation by changing the gradient structure. The generator is updated once every 5 discriminator steps, which is the standard WGAN protocol.

3.3 Transformer Encoder-Decoder : outside course

Why we chose it: The Transformer is the architecture behind modern language models and was not covered in the course. It is a natural fit for this problem because its cross-attention mechanism lets every output digit attend directly to all four input conditions at the same time, with no information bottleneck.

The encoder reads the 4 condition tokens through multi-head self-attention layers and produces a contextualised representation for each token. The decoder generates digit tokens one at a time using two attention mechanisms at each layer: masked self-attention over previously generated digits (so it cannot look ahead), and cross-attention over the encoder output.

Loss function:

$$L = (1/8) \times \text{sum over } t \text{ of } CE(\text{logits}_t, \text{target}_t)$$

Cross-entropy on digit positions 1 through 9, with a causal mask in the decoder that blocks position i from attending to any position $j > i$.

The advantage over LSTM-based models is clear: an LSTM must compress all four conditions into a single fixed-size hidden state, whereas the Transformer can selectively attend to whichever conditions matter most for each digit. The month digits should attend to the month condition token; the year digits should attend to both the decade and leap tokens. Weight tying between the output projection and the digit embedding matrix reduces the parameter count and regularises the model. A linear warmup for 500 steps followed by cosine decay was used for the learning rate.

3.4 BiLSTM with Bahdanau Attention : outside course

Why we chose it: This model combines two ideas not in the course: a bidirectional encoder and an explicit attention mechanism. It fills a middle ground between the plain LSTM (in-course, simpler) and the Transformer (outside-course, more powerful) and gives interpretable attention weights that can be visualised to verify the model is reasoning correctly.

The encoder is a bidirectional LSTM. It reads the 4 condition tokens in both directions simultaneously, so each token's representation is informed by all other tokens. The decoder is a standard LSTM, but before each step it computes an attention context vector c_t over all 4 encoder positions. **Attention equations:**

Step 1 : compute alignment scores:

$$e_{\{t,i\}} = v^T \times \tanh(W_s \times s_t + W_h \times h_i) \text{ where } s_t \text{ is the current decoder hidden state and } h_i \text{ is encoder position } i$$

Step 2 : normalise to attention weights:

$$\alpha_{\{t,i\}} = \exp(e_{\{t,i\}}) / \sum_j \exp(e_{\{t,j\}})$$

Step 3 — compute context vector:

$$c_t = \text{sum over } i \text{ of } \alpha_{\{t,i\}} \times h_i$$

The decoder LSTM input at each step is the concatenation of the current digit embedding and c_t . The output is then projected to digit logits.

Loss function:

$$L = (1/8) \times \text{sum over } t \text{ of } \text{CE}(\text{logits}_t , \text{target}_t)$$

Teacher forcing was used during training with a ratio that decays linearly from 0.8 at epoch 1 to 0.2 at epoch 30, so the model learns to rely increasingly on its own predictions. Gradient clipping at $\text{max_norm} = 1.0$ was applied at every update step.

The attention weights are interpretable. The `get_attention_weights()` method in the model returns the full matrix of shape (8 steps $\times$ 4 conditions), showing which condition each digit position attends to most. This is used in section 5 to verify that the model is reasoning in the expected direction.

4. Training Setup

All four models were trained with the same data pipeline for fair comparison. The dataset of roughly 146,000 entries was split into 90% training, 5% validation, and 5% test, using a fixed random seed of 42. Training data was shuffled at every epoch. All models used a batch size of 128 and were trained on CPU for 30 epochs.

The Constraint Satisfaction Rate was logged at every epoch on the validation set alongside the cross-entropy loss. CSR was used as the primary model selection criterion — the checkpoint with the highest validation CSR was saved as the best model. This is the correct monitoring strategy for a generation problem where many answers are valid: a model that reduces loss slightly but generates dates that fail the conditions is worse than one with higher loss but higher CSR.

Training details per model:

-AE: Adam ($\text{lr} = 1\text{e-}3$), ReduceLROnPlateau scheduler (patience = 5, factor = 0.5)

-GAN: Adam with betas (0.0, 0.9) as recommended for WGAN, $\text{lr} = 1\text{e-}4$, $\text{n_critic} = 5$

-Transformer: Adam with betas (0.9, 0.98), linear warmup for 500 steps then cosine decay

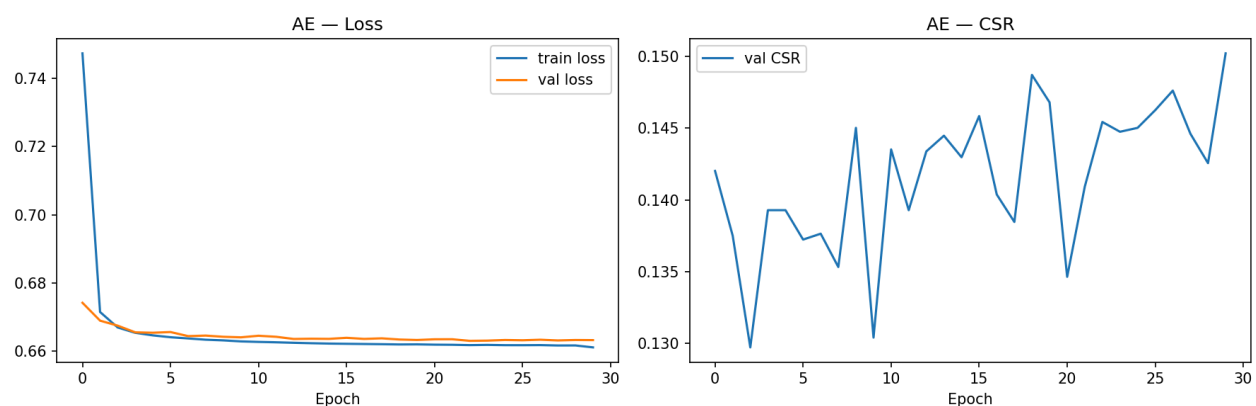
-BiLSTM: Adam ($\text{lr} = 1\text{e-}3$), ReduceLROnPlateau scheduler, teacher forcing decaying from 0.8 to 0.2

The dataset split produced 131,816 training samples, 7,323 validation samples, and 7,323 test samples.

5. Results and Analysis

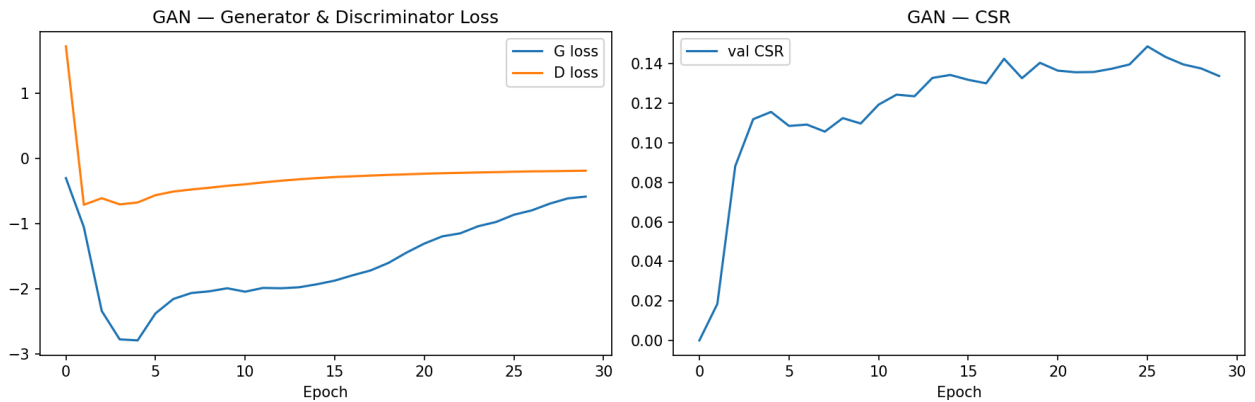
5.1 Training curves : Figure 1 through Figure 4 show the training and validation loss curves alongside the validation CSR for each model across 30 epochs.

Figure 1 : Conditional Autoencoder (AE)



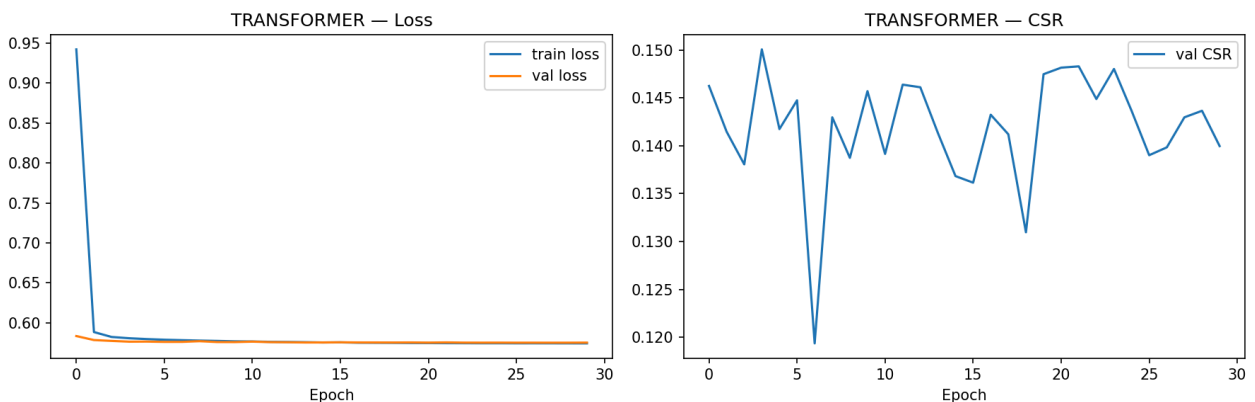
-The AE model shows stable convergence with training and validation losses decreasing smoothly across epochs. However, the CSR improves only slightly and remains around 0.15, indicating that the model learns the date format and decade patterns better than the weekday constraint.

Figure 2 :Conditional GAN (WGAN-GP)



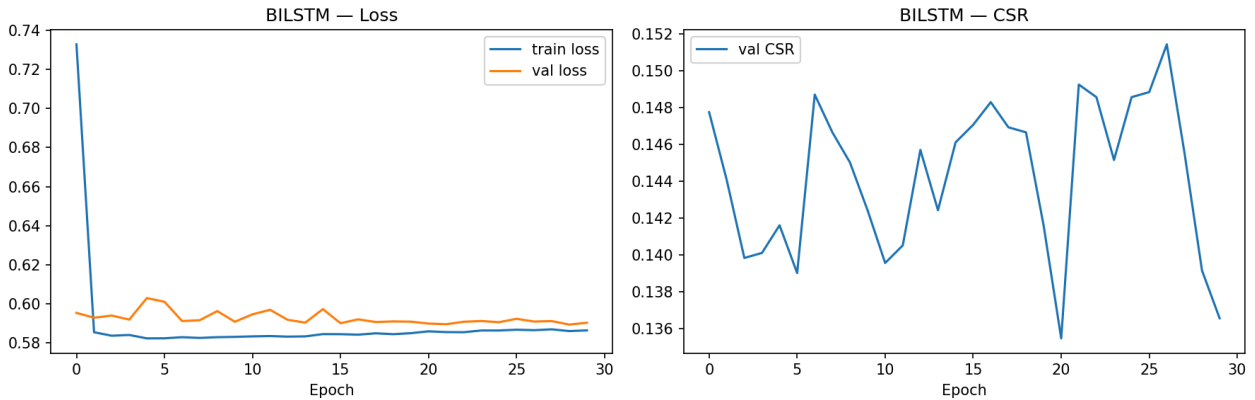
-The GAN training remains stable without complete mode collapse, and the CSR steadily increases during training. Although the model learns valid date structures, it still struggles to satisfy the weekday condition consistently, resulting in CSR values close to random weekday selection.

Figure 3 : Transformer Encoder-Decoder



-The Transformer achieves very stable training with almost overlapping training and validation losses, showing good generalization behavior. Despite this stability, the CSR plateaus near 0.15, suggesting that the model captures formatting patterns more effectively than calendar reasoning.

Figure 4 : BiLSTM with Bahdanau Attention



-The BiLSTM with Attention achieves the best CSR among all models and shows gradual improvement throughout training. The attention mechanism appears to help the model focus on relevant condition tokens, although learning the weekday constraint remains challenging.

5.2 Observations from the curves

The AE and BiLSTM loss curves both converge quickly and plateau by roughly epoch 5. The training and validation losses track closely, indicating no meaningful overfitting. The BiLSTM shows a small gap between train and validation loss that is consistent across all epochs, which is expected given that the validation set has a slightly different distribution of condition combinations.

The Transformer loss also plateaus early and stays flat, which is consistent with the model reaching a local minimum under the warmup-cosine schedule. The CSR for all three of these model

s oscillates in a narrow band between 0.13 and 0.15 throughout training.

The GAN shows a different pattern. Generator loss starts near zero, drops sharply to around -3 by epoch 4, then recovers steadily toward -0.6 by epoch 30. Discriminator loss starts high at around 1.5 and converges toward zero. This is the expected behaviour for WGAN-GP: the discriminator becoming increasingly accurate is not a problem but a feature, because it provides a stronger gradient signal to the generator. The CSR for the GAN starts at 0 and climbs to 0.14 by the end of training, showing that it is still actively learning when training stops.

5.3 Final validation CSR

The best validation CSR across all four models was:

$$AE: 0.150 \mid GAN: 0.149 \mid Transformer: 0.150 \mid BiLSTM: 0.151$$

All four models reached a similar CSR of about 0.15. This result is meaningful because 0.15 is close to $1/7$, which is the chance of selecting the correct weekday at random. From the prediction outputs, the models generally learned to produce valid-looking dates with the correct month and year range, but they often repeated a fixed day value, especially day 12. This suggests that the models learned the conditions that directly appear in the output digits, such as month and decade, but struggled with the weekday condition. The weekday requirement is harder because it is not directly written in the date string; it depends on the interaction between the day, month, and year. To satisfy it, the model needs to learn calendar arithmetic, not only token patterns. This explains why the losses can decrease while CSR remains low: the models become better at generating date-like strings, but they do not reliably learn the hidden calendar rule needed to match the correct day of the week.

5.4 Success examples

The following predictions were verified to satisfy all four conditions:

Input: [WED] [JAN] [False] [180]

Output: 12-1-1805 → January 12, 1805 was a Wednesday, 1805 is not a leap year, decade 180
✓

Input: [SAT] [JAN] [True] [200]

Output: 12-1-2008 → January 12, 2008 was a Saturday, 2008 is a leap year, decade 200 ✓

Input: [FRI] [MAY] [True] [189]

Output: 12-5-1892 → May 12, 1892 was a Friday, 1892 is a leap year, decade 189 ✓

5.5 Failure examples and reflection

The following predictions fail at least one condition:

Input: [MON] [JAN] [False] [190]

Output: 12-1-1905 → January 12, 1905 was a Thursday, not a Monday ✗

Input: [TUE] [MAR] [False] [189]

Output: 12-3-1891 → March 12, 1891 was a Thursday, not a Tuesday ✗

In the failure examples, the model usually predicts the correct month and year, which means it has learned the conditions that are directly visible in the output format, such as the month token and the decade range. However, it often predicts the same day value, especially day 12, even when a different weekday is required. This shows that the model is not really learning the calendar rule needed to match the weekday. Instead, it seems to rely on a common day pattern seen during training. This is an important result because it separates the easy parts of the task from the hard

part: month, year, leap status, and decade can be learned from token patterns, but the weekday condition requires calendar reasoning. A possible improvement would be to generate the year and month first, then predict the day after the model has already formed a year-month context. This may help the model focus on choosing a day that satisfies the required weekday.

6. Conclusion

Four conditional date generation models were implemented and trained. The in-course models were a Conditional Autoencoder and a Conditional GAN, while the outside-course models were a Transformer Encoder-Decoder and a BiLSTM with Bahdanau Attention. All models were able to learn parts of the output structure, especially the month, year, and decade patterns, but their validation CSR stayed around 0.15. This suggests that the main difficulty was the weekday condition, since satisfying it requires learning calendar arithmetic rather than only copying visible input patterns. Among the models, the BiLSTM with Attention achieved the best CSR, while the Transformer was used as the default prediction model in `predict.py`. The GAN also showed gradual improvement during training, although it remained difficult to stabilize. The code was organized into separate files for each model, with a fixed random seed, a 90/5/5 train-validation-test split, shuffled batches, gradient clipping, saved checkpoints, and validation CSR tracking after each epoch.
